Pre-training, Fine-tuning, and Prompting Strategies for Bug Fixing

Author: Abigail Schwall
Course: GenAI for Software Development
Professor: Antonio Mastropaolo

1 Abstract

This report presents an evaluation of the effects of pretraining on the finetuning results of a machine learning model designed to fix buggy java code. The results of both finetuning pipelines are then evaluated against a RAG model and zero-shot designed for the same purpose. RAG proved to be the most accurate model out of all 4 configuration with a CodeBLEU score of 0.2624.

2 Tokenizer Training and Model Configuration

The tokenizer used for this project is a SentencePiece tokenizer. The tokenizer was initialized with `model_type='unigram'` and `vocab_size=16384`. The output from tokenizer training was saved in a folder within the project directory named `t5_tokenizer` so it could be used for pretraining and finetuning.

3 Pretraining

The dataset used for pretraining was `~50,000` Java methods from CodeSearchNet taken from HuggingFace. The data was filtered to only use methods 10-512 tokens in length. The `SpanCorruptionDataset` class and `span_corruption` and `collate_fn` functions were taken from the lab notebook. The pretraining model was initialized as a small architecture `T5Config` model with 6 encoder layers, 6 decoder layers, 512-dimensional embeddings, and 8 attention heads. Pretraining was carried out over 3 epochs, and the training loss decreased with each epoch. The pretrained model was saved in a folder within the project directory named `pretrained_t5` so it could be used in pipeline A.

```
Dataset: 50000 samples, max_length=512
Starting 3-epoch pretraining...
Epoch 1 — Loss: 0.6640
Epoch 2 — Loss: 0.5568
Epoch 3 — Loss: 0.5424
Writing model shards: 100% 1/1 [00:00<00:00,  2.90it/s]
Model saved to pretrained_t5/
```

4 Finetuning

Both pipelines were finetuned using the buggy jave code dataset from `CodeXGlue` and were finetuned over 3 epochs. For evaluation, I used a smaller subset of 500 tests samples to be consistent with my RAG model (see RAG section below), but I got the same results when running evaluation on the larger test set.

4.1 Pipeline A - With pre-training

The model and tokenizer for pipeline A were loaded from `pretrained_t5`. Over the 3 epoch training, the loss did not significantly decrease overall, with only a 0.0004 difference between the 1st and 3rd epoch. The loss even increased between epochs 1 and 2.

```
Starting 3-epoch finetuning...
Epoch 1 — Loss: 0.0348
Epoch 2 — Loss: 0.0392
Epoch 3 — Loss: 0.0344
Writing model shards: 100% 1/1 [00:00<00:00,  1.25it/s]
```

4.2 Pipeline B - Without pre-training

The model in pipeline B was initialized with the same `T5Config` as the pre-training pipeline and the tokenizer was loaded from `t5_tokenizer`. Over the 3 epoch training, the loss steadily decreased with a 0.0036 difference between the 1st and 3rd epoch.

```
Starting 3-epoch finetuning...
Epoch 1 — Loss: 0.0363
Epoch 2 — Loss: 0.0342
Epoch 3 — Loss: 0.0327
Writing model shards: 100% 1/1 [00:00<00:00,  2.65it/s]
```

4.3 Exact match evaluation results

Both pipelines produced 0.0 exact matches.

4.4 BLEU evaluation results

Both pipelines produced a BLEU score of 0.25. This was surprising as I expected pipeline B - finetuned only to perform better given the training results.

```
=== RESULTS ===
Finetuned Pretrained Exact Match: 0.0000
Finetuned Pretrained CodeBLEU:   0.2500
Finetuned Only EM:               0.0000
Finetuned Only CodeBLEU:         0.2500
```

5 RAG

I had some issues with my RAG model training and ended up decreasing the test sample size to 500 in order to get my results in time.

5.1 RAG and zero-shot exact match results

Both RAG and zero-shot produced 0.0 exact matches.

5.2 RAG and zero shot CodeBLEU match results

The RAG model produced a CodeBLEU score of 0.2624 while zero-shot produced a CodeBLEU score of 0.2080. The RAG model has about a 26% relative increase in CodeBLEU score from the model that was tested with zero-shot. While both of these scores are relatively low, I believe they would be higher when run on the entire test set.

```
=== RESULTS ===
RAG Exact Match: 0.0000
RAG CodeBLEU:    0.2624
Zero-shot EM:    0.0000
Zero-shot CodeBLEU: 0.2080
```

6 Conclusion

Out of all four configuration, the RAG model performed the best with a CodeBLEU score of 0.2624. Both finetuning pipelines performed better than zero-shot, but I think this could have been due to the difference in test set size. I believe that if run on the full test set, both RAG and zero-shot CodeBLEU scores would be higher.

Pretraining did not effect the accuracy of my pretrained model for this experiment. Both pipelines had the same CodeBLEU score of 0.25.

All 4 pipelines produced 0.0 exact matches.

Repository: https://github.com/amschwall/RAG_Model